

enfys_mica_sampling

February 19, 2025

1 Simulating Spectral Sampling of the MICA Files with Enfys

In this notebook we simulate the spectral resolution of Enfys, sample the MICA Files spectral library, remove the continuum, and illustrate this sampling with a spectrogram.

2 Overview

1. Load MICA Files High-Resolution library.
2. Build Enfys simulator instrument
3. Sample MICA with Enfys
4. Produce Visualisations

3 Setup

```
[33]: %load_ext autoreload
      %autoreload 2
      %matplotlib inline
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

```
[34]: import sptk.config as cfg
```

We set our simulation resolution range to extend the range of Enfys, of 0.9 - 3.1 μm , such that the long tails of the Cauchy distribution of the linear-variable-filter profiles can be accommodated during sampling.

```
[36]: cfg.update_sample_res('wvl_min', 850)
      cfg.update_sample_res('wvl_max', 3300)
```

```
[37]: PROJECT_NAME = 'enfys_mica'
```

4 Data

4.1 Spectral Library

We use the MICA Files example spectral library to illustrate the sampling of key minerals identified on Mars.

```
[38]: from sptk.material_sets import MICA_SET
```

```
[39]: from sptk.material_collection import MaterialCollection
```

We make our high-resolution ‘material collection’ as an object that hosts this spectral library, and that provides a set of methods for visualising and handling it.

Note that here we’ve set the `allow_out_of_bounds` flag to `True`, which means that if a spectral library entry doesn’t completely span the spectral range, then we will still include it and plot the subset of the range that it does cover.

```
[8]: matcol = MaterialCollection(  
    MICA_SET, # defines the material classes and their mineral group  
    ↳ constituents  
    'MICA_new_dirtree', # the spectral library directory to use  
    PROJECT_NAME, # the name of the project  
    load_existing=False, # if True, load an existing Material Collection  
    ↳ from the project directory  
    balance_classes=False, # if True, ensure equal number of entries in  
    ↳ each class  
    random_bias_seed=None, # if set to an integer this ensures repeatable  
    ↳ class balancing  
    allow_out_of_bounds=True, # if False, discard entries of the spectral  
    ↳ library that don't span the spectral range  
    plot_profiles=False, # if true, plot profiles during construction  
    export_df=True) # if true, save the output for later access during  
    ↳ construction
```

Building new enfys_mica MaterialCollection DF

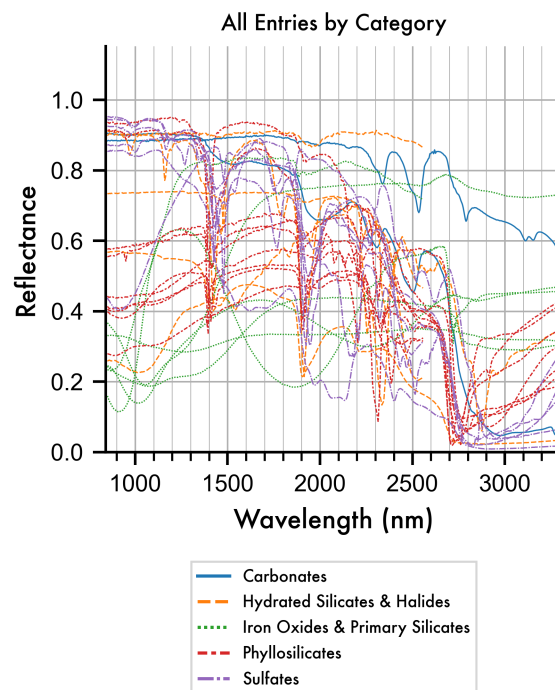
```
Iron Oxides & Primary Silicates: NBPP22.csv (Clinopyroxene) loaded  
Iron Oxides & Primary Silicates: LAPP47B.csv (Orthopyroxene) loaded  
Iron Oxides & Primary Silicates: LAP005.csv (Fayalite) loaded  
Iron Oxides & Primary Silicates: LASC02.csv (Forsterite) loaded  
Iron Oxides & Primary Silicates: GDS27.csv (Hematite) loaded  
Iron Oxides & Primary Silicates: NCLS04.csv (Plagioclase) loaded  
Sulfates: F1CC08B.csv (Alunite) loaded  
Sulfates: GDS145.csv (Bassanite) loaded  
Sulfates: LASF41A.csv (Gypsum) loaded  
Sulfates: LASF21A.csv (Jarosite) loaded  
Sulfates: F1CC15.csv (Kieserite) loaded  
Sulfates: 799F366.csv (Mg-Sulphate) loaded  
Phyllosilicates: LACL14.csv (Chlorite) loaded  
Phyllosilicates: LAKA04.csv (Kaolinite) loaded  
Phyllosilicates: LAILO2.csv (Illite) loaded  
Phyllosilicates: GDS106.csv (Margarite) loaded  
Phyllosilicates: LALZ01.csv (Serpentine) loaded  
Phyllosilicates: LAM002.csv (Montmorillonite) loaded
```

Phyllosilicates: NCJB26.csv (Nontronite) loaded
 Phyllosilicates: LASA58.csv (Saponite) loaded
 Phyllosilicates: GDS23.csv (Talc) loaded
 Carbonates: CAGR01.csv (Calcite) loaded
 Carbonates: F1CC06B.csv (Magnesite) loaded
 Hydrated Silicates & Halides: HS433.csv (Halite) loaded
 Hydrated Silicates & Halides: LAZE03.csv (Prehnite) loaded
 Hydrated Silicates & Halides: bkrijb329c.csv (Siliceous-Sinter) loaded
 Hydrated Silicates & Halides: GDS26.csv (Epidote) loaded
 Hydrated Silicates & Halides: GDS1.csv (Analcime) loaded
 Loading 28 of entries complete.
 Exporting the Material Collection to CSV and Pickle formats...

We can visualise the content of the MaterialCollection through several plotting options.

We can plot all entries using the actual reflectance values:

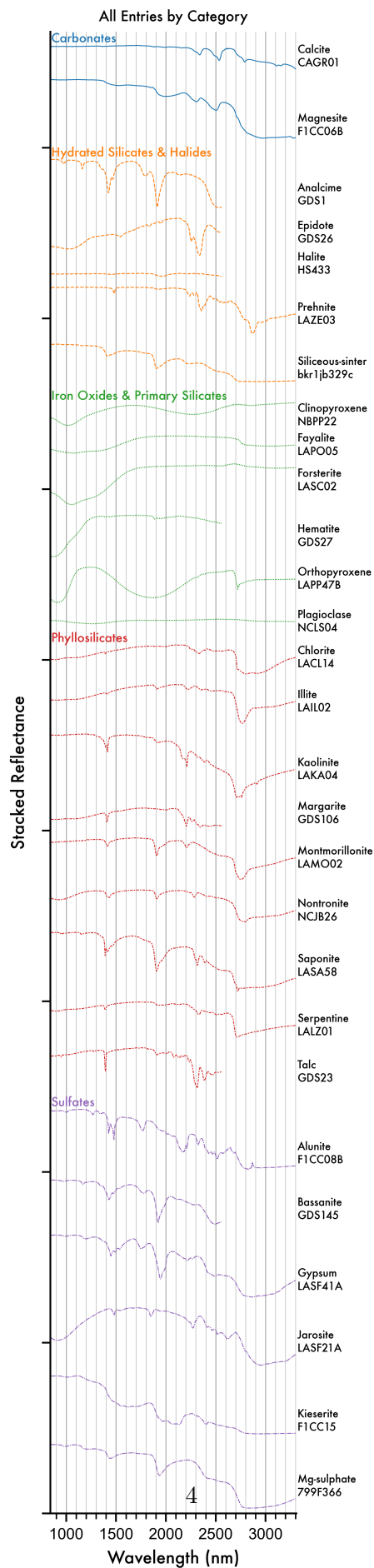
```
[9]: ax = matcol.plot_profiles(scope='all', groupby='Category')
```



However, this is difficult to interpret, due to overlapping profiles.

Instead, we can produce a stacked plot, with the 'stacked' keyword:

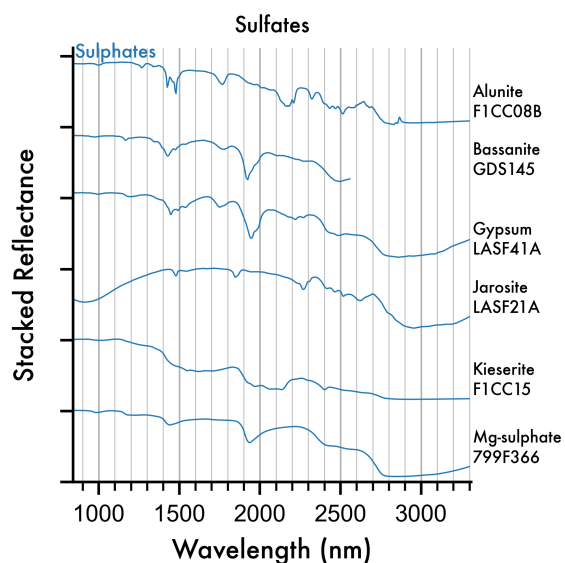
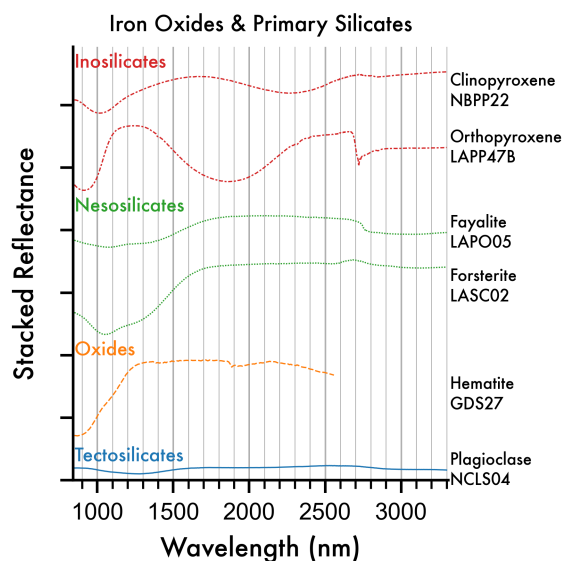
```
[10]: ax = matcol.plot_profiles(scope='all', groupby='Category', stacked=True)
```

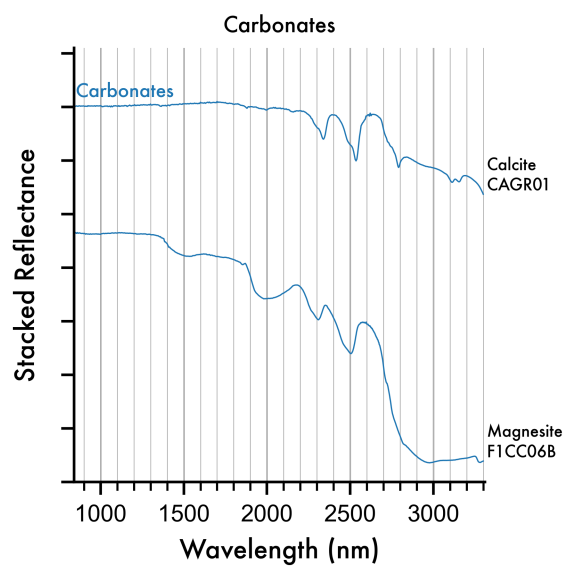
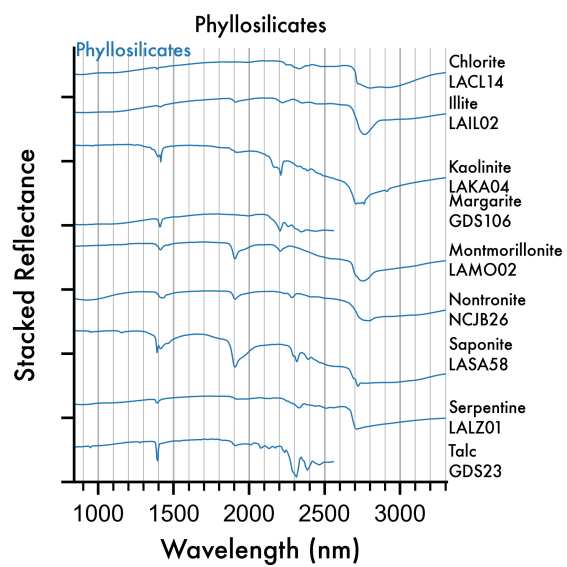


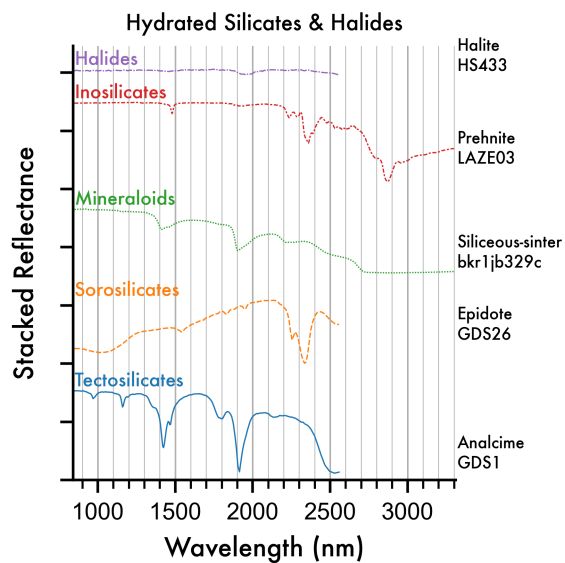
This distributes the data with offsets to avoid overlap. The data can be grouped according to the ‘groupby’ keyword.

As this plot is long, we can break it down by using the ‘scope’ keyword, to, e.g., produce a separate plot for each Category. It may then be interesting to further breakdown each category plot into the mineral groups, by passing ‘Group’ to the ‘groupby’ keyword.

```
[11]: ax = matcol.plot_profiles(scope='categories', groupby='Group', stacked=True)
```

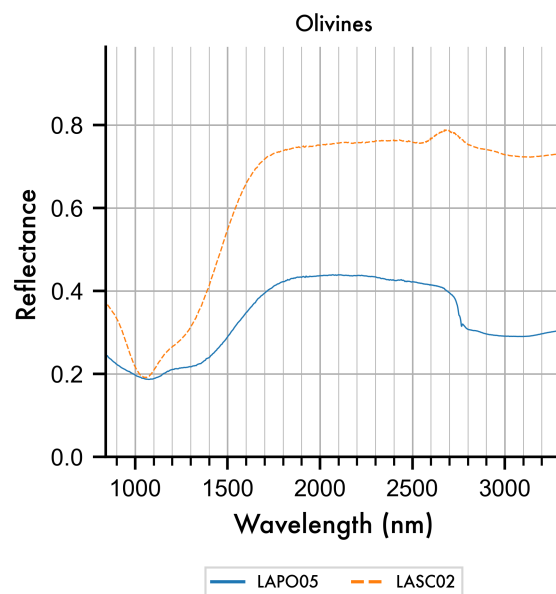






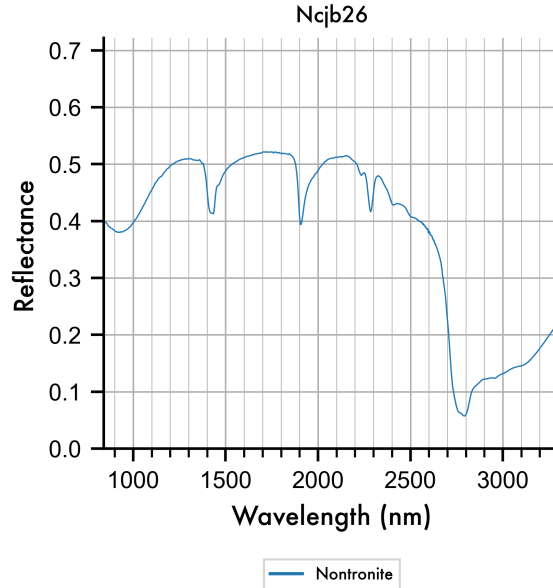
We can select specific groupings by passing specific words to the 'scope' argument. For example, we can find all materials that belong to the olivine subgroup:

```
[12]: ax = matcol.plot_profiles(scope='olivines', groupby='Sample ID', stacked=False)
```



Or even select a single entry:

```
[13]: ax = matcol.plot_profiles(scope='NCJB26', groupby='Species', stacked=False)
```



4.2 Instrument

To simulate Enfys, rather than providing a file with the expected spectral response of each position of the Linear-Variable Filter (LVF, above), we specify the spectral range and resolution and sampling regime, and generate a set of central-wavelengths and bandwidths. We do this with the `InstrumentBuilder` class of the `sptk.instrument` module.

```
[14]: from sptk.instrument import InstrumentBuilder
```

The expected spectral resolution of Enfys is $\lambda/\Delta\lambda = 100$, and the range is 0.9 - 3.1 μm .

```
[15]: enfys_builder = InstrumentBuilder(
    instrument_name='enfys',
    instrument_type='lvf',
    sampling='nyquist',
    resolution = 100,
    spectral_range=[900, 3101],
    snr_range = [2000, 100])
```

Exporting instrument to ../data/instruments/enfys.csv...

```
[16]: from sptk.instrument import Instrument
```

Now we build the transmission profiles for each LVF position, according to the list of CWLs and FWHMs generated above. These are accessed by reading the `.csv` file produced by the `InstrumentBuilder` procedure.

When generating the profiles, we turn the CWL and FWHM into a profile by assuming a distribution function. For Enfys we use the Cauchy distribution, which behaves similarly to a Gaussian, but with

wider ‘wings’. It is parameterised by the full-width-at-half-maximum, as the distribution strictly has no finite variance. We have implemented a `build_cauchy_filter` method for the `Instrument` class.

```
[17]: enfys = Instrument(
        name = 'enfys', # filename of the instrument information, held in /
        ↪software/data/instruments/
        project_name = matcol.project_name, # the project directory name,
        ↪hosting outputs in the /spectral_parameter_studies/ directory
        shape = 'cauchy',
        load_existing=False, # if True, load existing instrument data from
        ↪the project directory
        plot_profiles=False, # if true, plot the transmission profiles
        ↪during construction
        export_df=True) # if True, export the instrument transmission
        ↪profiles to the project directory
```

Building Instrument...

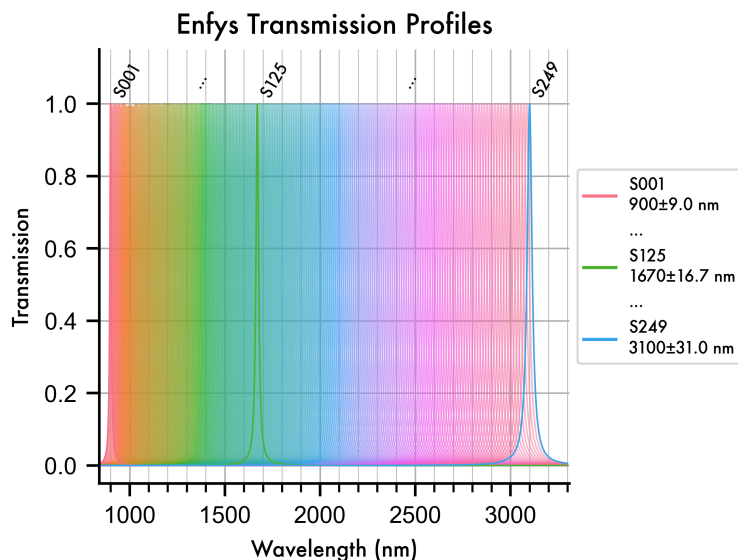
Building new DataFrame for 'enfys'...

Exporting the Instrument to CSV and Pickle formats...

```
[18]: fig, ax = enfys.plot_filter_profiles()
```

Plotting Instrument Transmission...

Plots exported to ../projects/enfys_mica/instrument

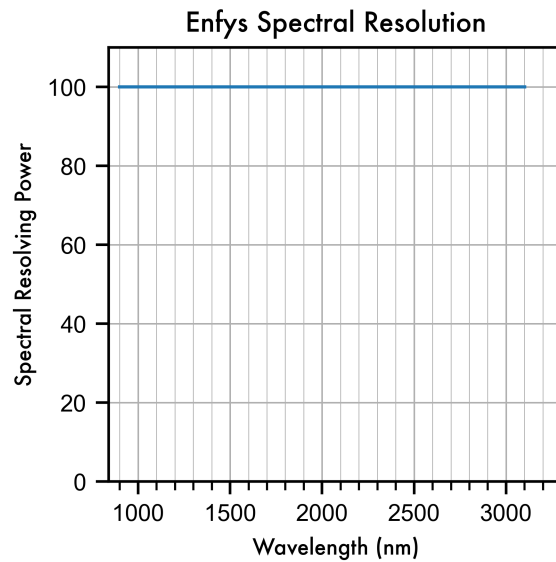


```
[19]: fig, ax = enfys.plot_spectral_resolution()
```

Plotting Spectral Resolution...

Plots exported to ../projects/enfys_mica/instrument

<Figure size 314.961x314.961 with 0 Axes>

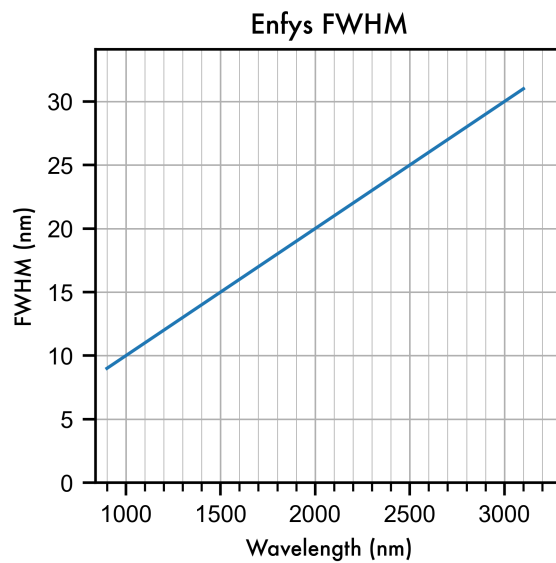


```
[20]: fig, ax = enfys.plot_fwhm()
```

Plotting FWHM...

Plots exported to ../projects/enfys_mica/instrument

<Figure size 314.961x314.961 with 0 Axes>

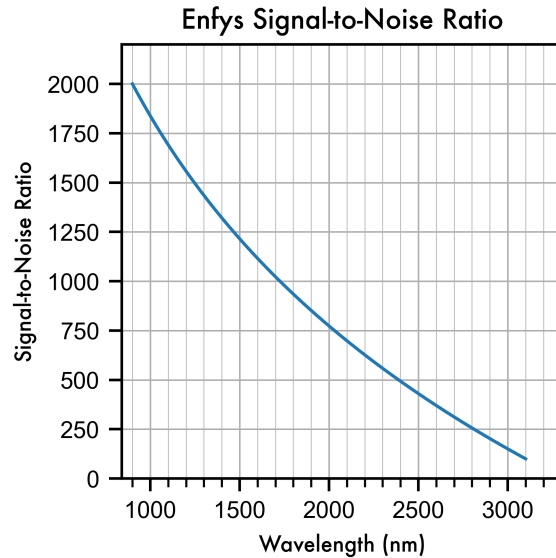


```
[21]: fig, ax = enfys.plot_snr()
```

Plotting Signal-to-Noise Ratio...

Plots exported to ../projects/enfys_mica/instrument

<Figure size 314.961x314.961 with 0 Axes>



This crude plot illustrates all of the linear-variable-filter positions considered here, under Nyquist sampling. That, the interval between each CWL is $\text{FWHM} / 2$. This gives 249 channels over 0.9 - 3.1 μm .

5 Sampling MICA with Enfys

We now make an Observation object by sampling the MICA files MaterialCollection with the Enfys Instrument.

```
[22]: from sptk.observation import Observation
```

```
[23]: obs = Observation(  
    material_collection=matcol,  
    instrument = enfys,  
    load_existing=False,  
    plot_profiles=False,  
    export_df=True)
```

Building new Observation DataFrame

for 'enfys_mica'...

Sampling the Material Collection with the Instrument...

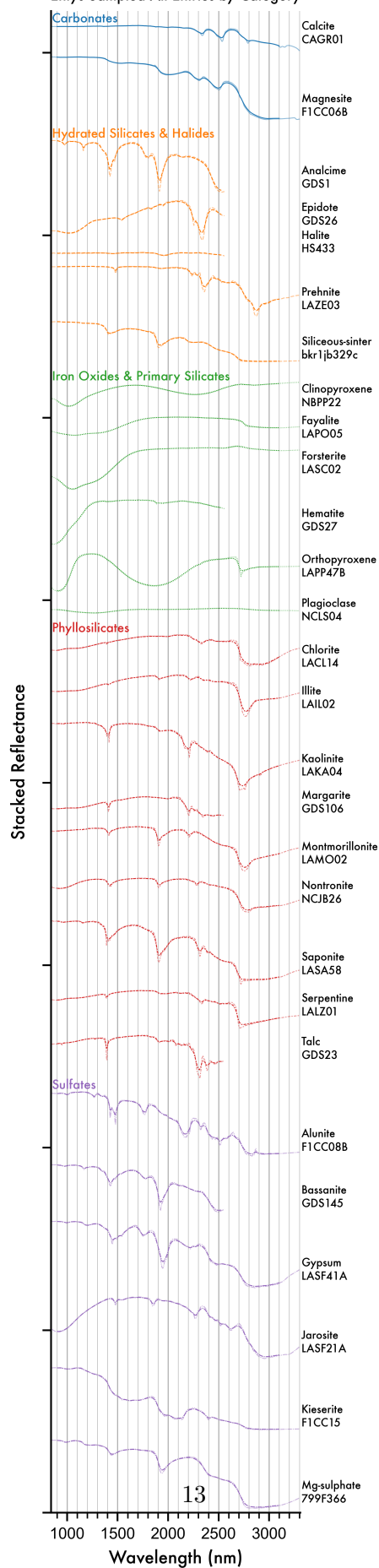
Sampling complete.

Exporting the Observation Pickle format...
Observation export complete.

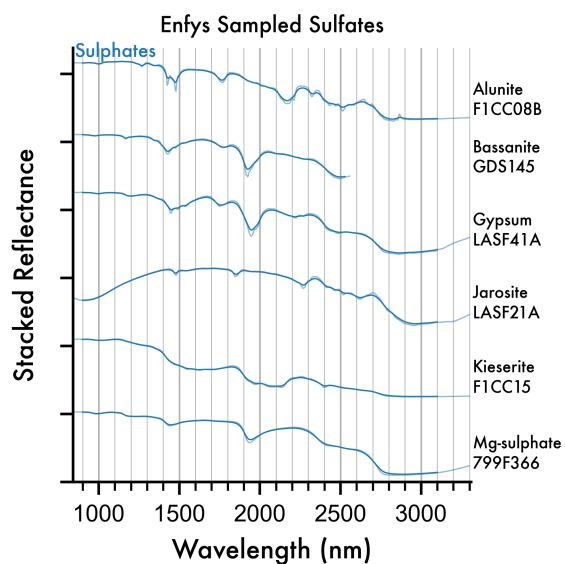
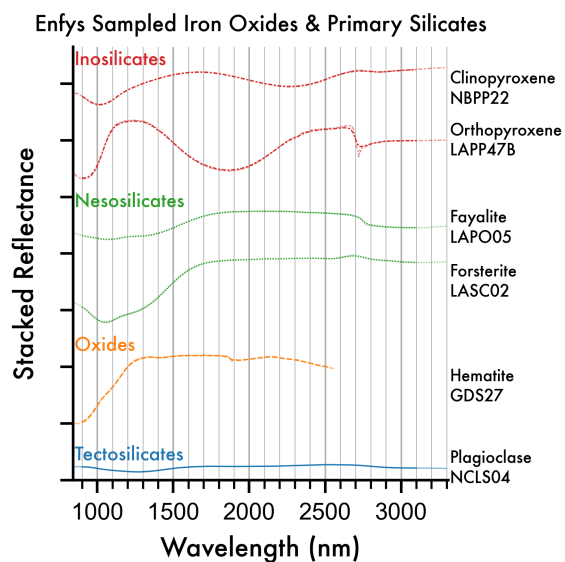
We can plot the sampled spectra against the input high-spectra below:

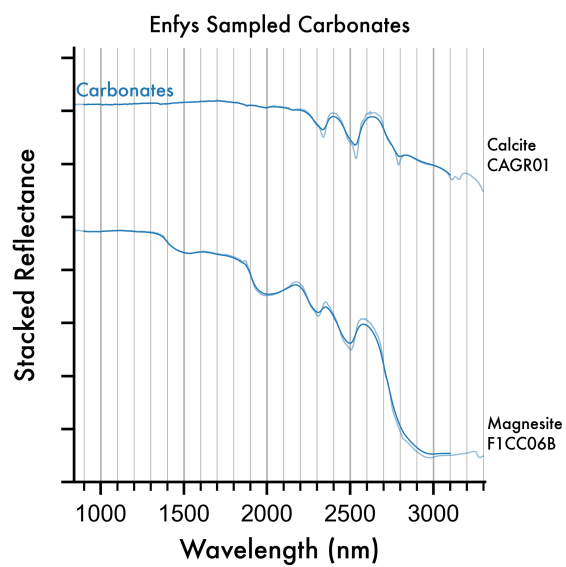
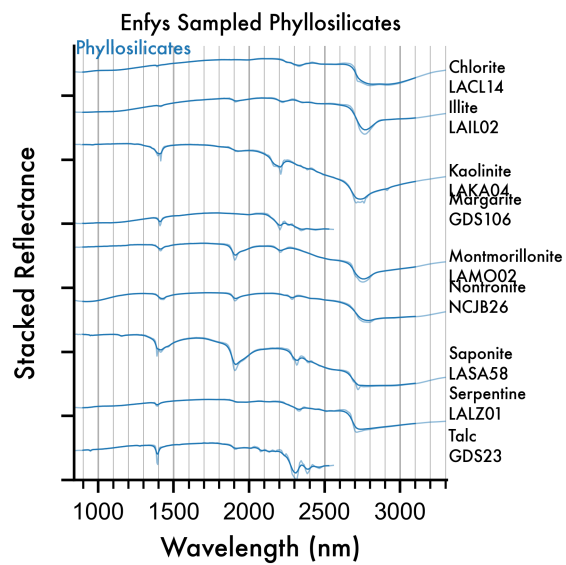
```
[24]: axes = obs.plot_profiles(scope='all', groupby='Category', stacked=True,   
    ↪ hires_under=True)
```

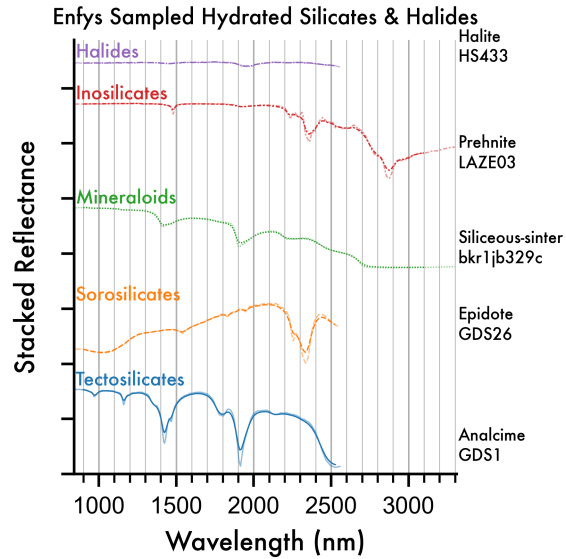
Enfys Sampled All Entries by Category



```
[25]: axes = obs.plot_profiles(scope='categories', groupby='Group', stacked=True,
    ↪ hires_under=True, pad_factor= 1/4)
```







To collect the spectra together in an easy to visualise way, we can also produce a ‘spectrogram’.

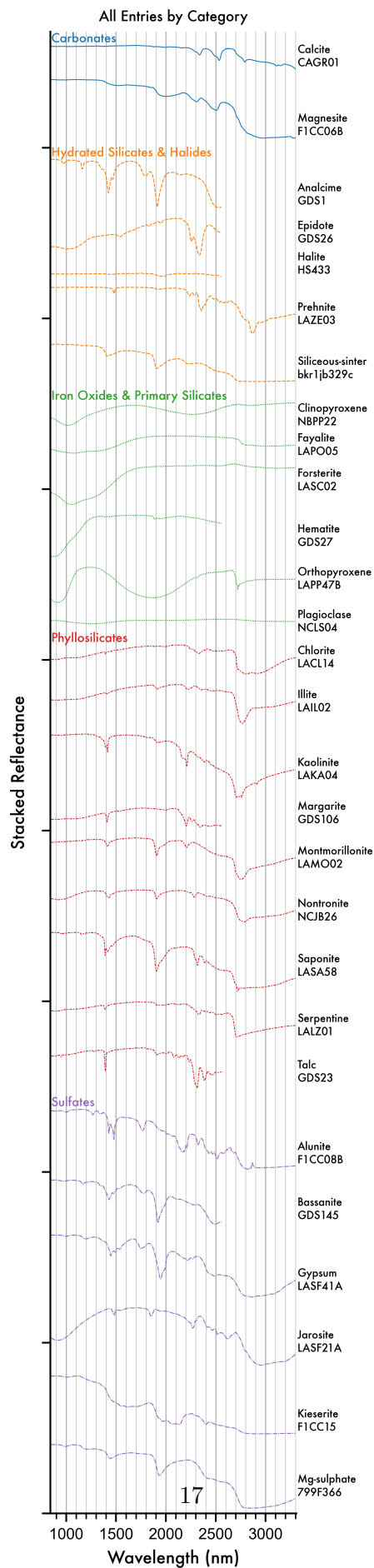
```
[26]: from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla
```

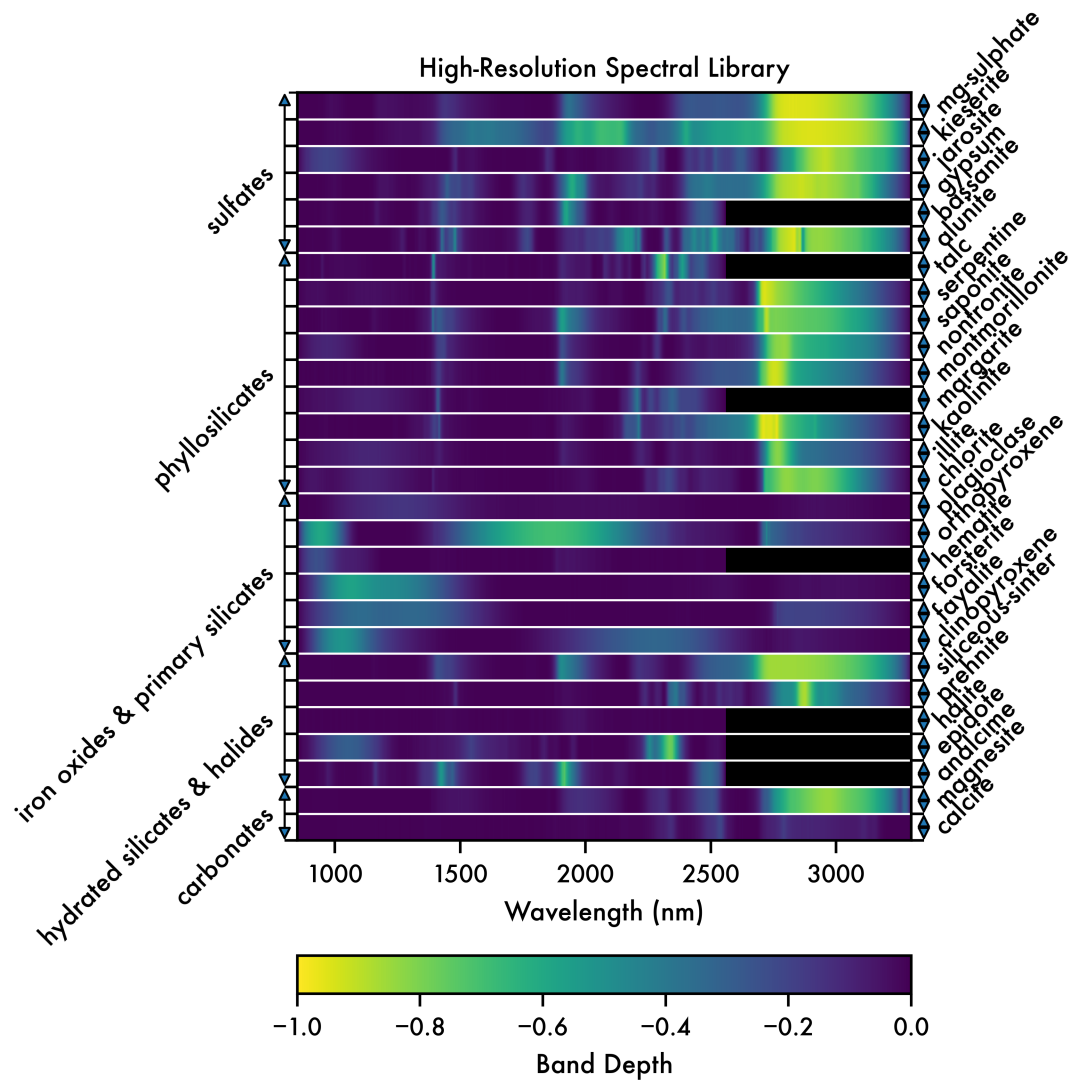
```
[27]: plottr_mc = sla(matcol)
```

```
[28]: cr_obj = plottr_mc.remove_continuum()
      plottr_mc.plot_profiles(stacked=True)
```

```
[28]: <AxesSubplot: title={'center': 'All Entries by Category'}, xlabel='Wavelength
      (nm)', ylabel='Stacked Reflectance'>
```

```
[29]: fig, ax = plottr_mc.visualise_spectrogram()
```





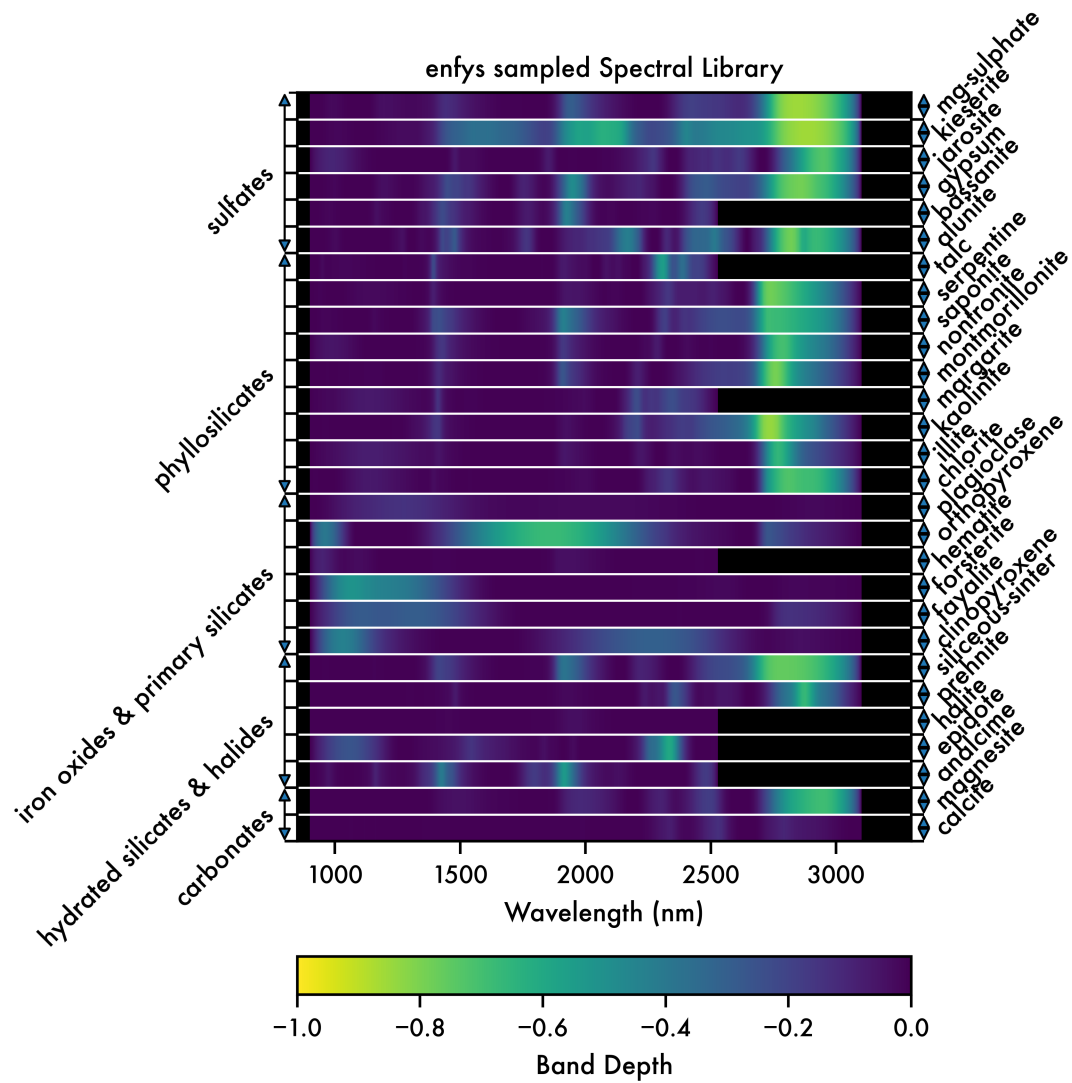
Now we look at the EnfyS sampled library:

```
[30]: plotter = sla(obs)
```

```
[31]: plotter.remove_continuum()
```

```
[31]: <sptk.observation.Observation at 0x30d518c10>
```

```
[32]: fig, ax = plotter.visualise_spectrogram()
```



By comparing the Material Collection high resolution spectrograph to the Enfys sampled spectrograph, we can get an idea of the resolution of Enfys.